Module 8: API Security and Access Control

Protecting APIs from unauthorized access, abuse, and data exposure

What You Will Learn

- ✓ Understand common API security risks
- ✓ Differentiate authentication and authorization
- ✓ Use HTTPS, OAuth2, API Keys, and WebAuthn
- ✓ Apply rate limiting, validation, and error handling
- ✓ Connect API security practices with OWASP risks

APIs Expose Data and Actions

- APIs connect clients to sensitive business logic
- They may expose users, payments, files, orders, or private data
- A weak API can lead to data leaks or unauthorized changes

Security must be part of the API design, not an afterthought

Security Works in Layers

Monitoring: Logs and alerts	
Safe output: Error handling	
Safe input: Validation	
Abuse control: Rate limiting	
Permissions: Authorization	
Identity: Authentication	
Secure transport: HTTPS	

Use HTTPS Everywhere

- ✓ Encrypts communication between client and server
- ✓ Protects passwords, tokens, cookies, and private data
- ✓ Prevents interception and tampering
- ✓ Never send credentials or tokens over plain HTTP

Clarify that HTTPS does not authenticate or authorize users by itself. It protects the communication channel and is the minimum requirement for any secure API.

Authentication: Who Are You?

- Confirms the identity of a user or application

Common methods:

Username and password

JWT access tokens

OAuth2

API Keys

WebAuthn / Passkeys

- Without authentication, the API does not know who is calling

Authorization: What Can You Do?

- Defines what an authenticated user is allowed to access.
- Permissions should be checked by:
  - Role
  - Resource
  - Action
  - Data owner

Being logged in does not mean having access to everything

```
GET /api/users/25/orders
```

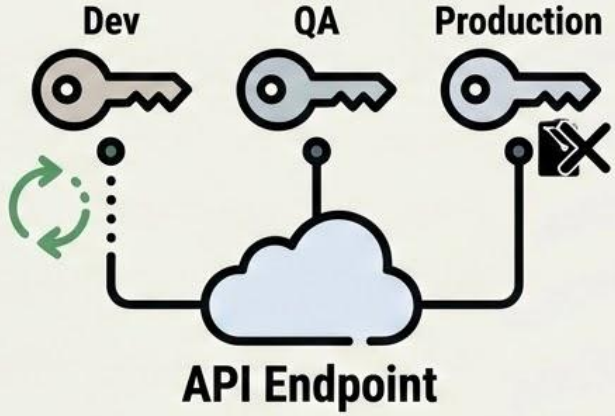
What happens if the user changes 25 to 26?

Can View	Cannot Modify
	
User 25's own orders	Other users' orders / data

Explain the difference between authentication and authorization. Authentication answers 'who are you?' Authorization answers 'what are you allowed to do?'

Use API Keys Carefully

- An API Key identifies an application or client
- It should not be the only protection for end users
- **Best practices**:
  - Use different keys per environment
  - Limit permissions
  - Rotate keys periodically
  - Revoke compromised keys
  - Never commit keys to GitHub

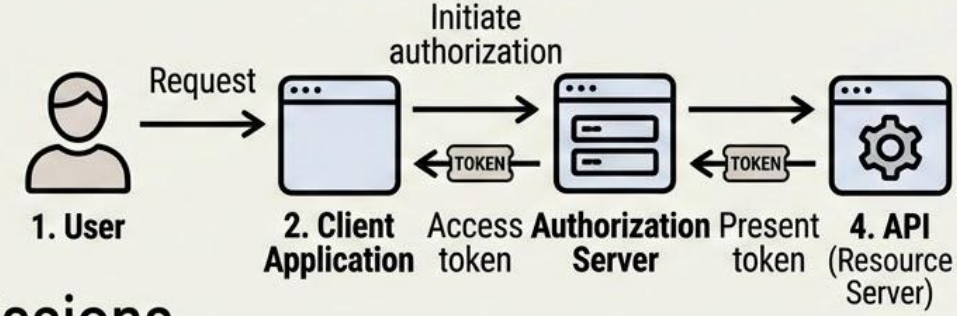


Explain that API Keys are useful for identifying applications, but they do not always identify a specific user. They should be protected like secrets.

OAuth2: Delegated Access

- Allows limited access without sharing passwords
- Main actors:
  - Resource Owner
  - Client Application
  - Authorization Server
  - Resource Server
- Uses scopes to limit permissions

  An app can request permission to read a profile, but not modify the full account.



Explain that OAuth2 is commonly used when one application needs to access resources on behalf of a user.

Access Tokens Represent Temporary Permission

- A token represents a temporary authorization

JWT Token Card

JSON Web Token (JWT)

- **sub**: **sub**: user identifier (e.g., specific user ID)
- **role**: **role**: user role (e.g., Admin)
- **scope**: **scope**: allowed actions (e.g., read, write)
- **exp**: **exp**: expiration time

- The backend must validate the token before trusting it
  - ✅ Signature
  - ✅ Expiration
  - ☐ Issuer
  - ✅ Audience
  - ☐ Permissions

Emphasize that having a valid token does not automatically mean the user can access every resource. Authorization checks are still required.

Slide 11: WebAuthn and MFA

Stronger Authentication with WebAuthn and MFA

- WebAuthn enables passkeys, biometrics, and security keys
- Reduces password and phishing risks
- The server stores a public key, not the user password
- MFA adds an extra security layer

Examples:

Fingerprint

Face ID

Security key

Authenticator app

Explain that modern systems are moving beyond passwords. WebAuthn and MFA make account takeover more difficult.

Rate Limiting Prevents Abuse

- Controls how many requests a client can make

Helps protect against:
- Brute force attacks
- Scraping
- Denial of service
- Resource abuse

Can be applied by:
- IP address
- User
- API Key
- Endpoint
- Sensitive action

Example:
Limit login attempts to 5 requests per minute per user or IP.

Explain that rate limiting is not just about performance. It is also a security control that reduces the impact of automated attacks.

Never Trust Client Input

Use DTOs and validation rules

- Validate every value received by the API
- Check:
  - Data type
  - Length
  - Format
  - Allowed range
  - Required fields
- Reject unexpected fields

Example:

- Do not allow the client to send: `` `sAdmin = true` ``

Explain that the client can be modified. Even if a field is hidden in the UI, an attacker can still send it directly to the API

Return Helpful but Safe Errors

Safe Error

Good errors:

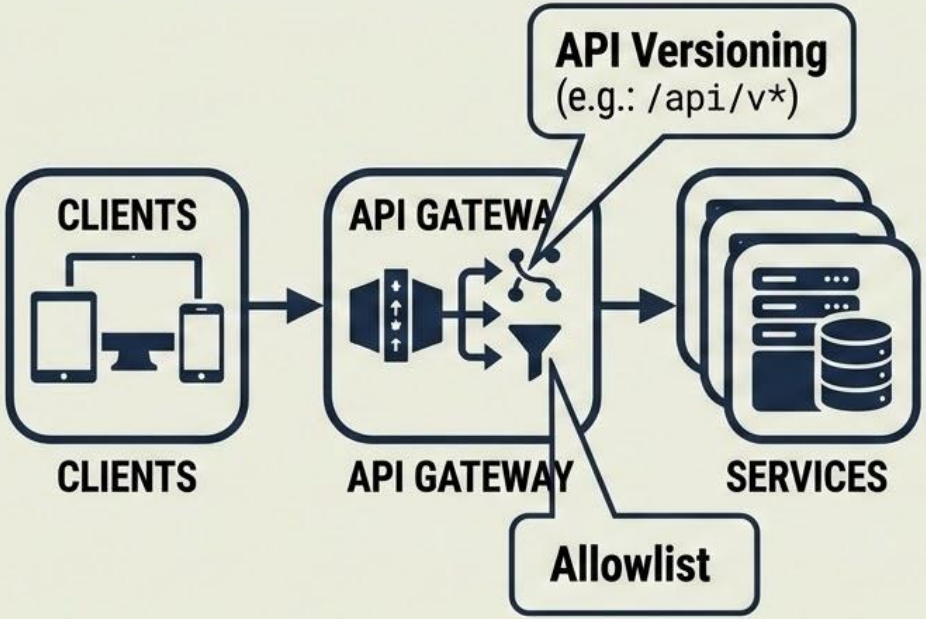
- Clear message
- Correct HTTP status code
- No internal details
- Useful for the client

Unsafe Error

- Bad errors:
  - Stack traces
  - SQL queries
  - Table names
  - Internal server paths
  - Incorrect status codes

Explain that error messages should help legitimate users and developers, but they should not reveal technical details that help attackers

Architecture Controls



- **API Gateway:** Centralizes routing, authentication, rate limiting, and logs
- **API Versioning:** Prevents
  - breaking existing clients
  - Allows old insecure versions to be retired
  - Example: `/api/v1/users`
- **Allowlist:** Restricts access to approved IPs, systems, or clients
  - Adds another layer of control

Explain that these controls help organize and protect APIs at the architecture level, but they do not replace proper authentication and authorization in the backend

Slide 16: OWASP API Security Risks

Common API Security Risks

Broken Object-Level Authorization

Broken Authentication

Excessive Data Exposure

Unrestricted Resource Consumption

Broken Function-Level Authorization

Unsafe Consumption of APIs

Improper Inventory Management

Future Risk

Explain that OWASP provides a useful list of common API risks. These risks appear frequently in real-world systems and should be considered during development and testing.

API Security Checklist

Before releasing an API, ask:

- ❑ Is HTTPS required?
- ❑ Is authentication implemented correctly?
- ❑ Is authorization checked per resource and action?
- ❑ Are inputs validated?
- ❑ Are errors safe and consistent?
- ❑ Is rate limiting enabled?
- ❑ Are sensitive actions logged?
- ❑ Are old API versions controlled or retired?
- ❑ Are secrets protected?

SECURED API

Use this slide as a practical checklist students can apply to their own projects.